

Lab 11 – Data Structures with AI: Implementing Fundamental Structures

Name: M.Dinesh

Roll no:2503a52374

Batch-13

Task 1 – Stack Implementation

1. Question

Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

2.Prompt:

Complete this Python Stack class for a beginner data structures lab. Add push(), pop(), peek(), and is_empty(), handle an empty stack, include short docstrings/comments, and add a small test section.

3. Code:

```
class Stack:
    """A simple Stack implementation using a Python list."""

    def __init__(self):
        """Initialize an empty stack."""
        self.items = []

    def push(self, item):
        """Add an item to the top of the stack."""
        self.items.append(item)

    def pop(self):
        """Remove and return the top item."""
        if self.is_empty():
            return None
        return self.items.pop()

    def peek(self):
        """Return the top item without removing it."""
        if self.is_empty():
            return None
        return self.items[-1]

    def is_empty(self):
        """Check whether the stack is empty."""
```

```
return len(self.items) == 0
```

```
# Testing the Stack
```

```
stack = Stack()
```

```
stack.push(10)
```

```
stack.push(20)
```

```
stack.push(30)
```

```
print("Stack:", stack.items)
```

```
print("Top element:", stack.peek())
```

```
print("Removed element:", stack.pop())
```

```
print("Stack after pop:", stack.items)
```

```
print("Is stack empty?", stack.is_empty())
```

4. Output :

```
[Running] python -u "c:\Users\Dines\OneDrive\Documents\ai assistant co
Stack: [10, 20, 30]
Top element: 30
Removed element: 30
Stack after pop: [10, 20]
Is stack empty? False
```

5. Final Justification

The Stack was implemented successfully with the required methods. It follows the LIFO principle and was tested with sample operations.

Task 2 – Queue Implementation

1. Question

Use AI to implement a Queue using Python lists.

2. Prompt:

Complete this Python Queue class using a list. Add enqueue(), dequeue(), peek(), and size(), follow FIFO, handle an empty queue, and add short comments plus a small test section.

3. Code

```
class Queue:
```

```
    """A simple Queue implementation using a Python list."""
```

```
    def __init__(self):
```

```
"""Initialize an empty queue."""
```

```
self.items = []
```

```
def enqueue(self, item):
```

```
    """Add an item to the rear of the queue."""
```

```
    self.items.append(item)
```

```
def dequeue(self):
```

```
    """Remove and return the front item."""
```

```
    if self.size() == 0:
```

```
        return None
```

```
    return self.items.pop(0)
```

```
def peek(self):
```

```
    """Return the front item without removing it."""
```

```
    if self.size() == 0:
```

```
        return None
```

```
    return self.items[0]
```

```
def size(self):
```

```
    """Return the number of items in the queue."""
```

```
    return len(self.items)
```

```
# Testing the Queue
```

```
queue = Queue()
```

```
queue.enqueue(10)
```

```
queue.enqueue(20)
```

```
queue.enqueue(30)
```

```
print("Queue:", queue.items)
```

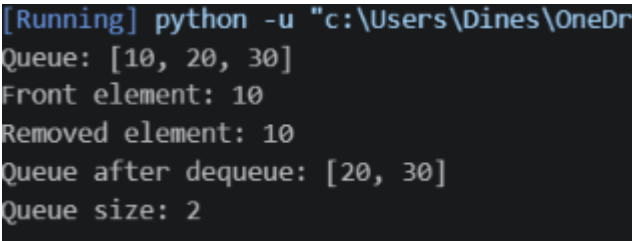
```
print("Front element:", queue.peek())
```

```
print("Removed element:", queue.dequeue())
```

```
print("Queue after dequeue:", queue.items)
```

```
print("Queue size:", queue.size())
```

4. Output Screenshot:



```
[Running] python -u "c:\Users\Dines\OneDr
Queue: [10, 20, 30]
Front element: 10
Removed element: 10
Queue after dequeue: [20, 30]
Queue size: 2
```

5. Final Justification

The Queue was implemented using a Python list and tested with sample operations. It follows the FIFO principle as required.

Task 3 – Linked List

1. Question

Use AI to generate a Singly Linked List with insert and display methods.

2. Prompt

Create a beginner-friendly singly linked list in Python with Node and LinkedList classes. Add insert() and display(), with short comments/docstrings and a small test section.

3. Code

class Node:

```
    """Represents one node in the linked list."""
```

```
    def __init__(self, data):
```

```
        """Initialize a node with data."""
```

```
        self.data = data
```

```
        self.next = None
```

class LinkedList:

```
    """A simple singly linked list."""
```

```
    def __init__(self):
```

```
        """Initialize an empty linked list."""
```

```
        self.head = None
```

```
    def insert(self, data):
```

```
        """Insert a new node at the end of the list."""
```

```
        new_node = Node(data)
```

```
        if self.head is None:
```

```
self.head = new_node
```

```
return
```

```
current = self.head
```

```
while current.next is not None:
```

```
    current = current.next
```

```
current.next = new_node
```

```
def display(self):
```

```
    """Display all elements in the linked list."""
```

```
    current = self.head
```

```
    while current is not None:
```

```
        print(current.data, end=" -> ")
```

```
        current = current.next
```

```
    print("None")
```

```
# Testing the Linked List
```

```
linked_list = LinkedList()
```

```
linked_list.insert(10)
```

```
linked_list.insert(20)
```

```
linked_list.insert(30)
```

```
print("Linked List:")
```

```
linked_list.display()
```

4. Output :

```
[Running] python -u "c:\Users\Dines\OneDrive\Documents\ai assistant coding\as  
Linked List:  
10 -> 20 -> 30 -> None
```

5. Final Justification

The singly linked list was implemented with nodes connected through next references. Insert and display operations were tested successfully.

Task 4 – Binary Search Tree (BST)

1. Question

Use AI to create a BST with insert and in-order traversal methods.

2. GitHub Copilot Prompt

Create a beginner-friendly Python BST with recursive insert() and in-order traversal. Add short comments/docstrings and test it with sample values.

3. Code

class Node:

```
    """Represents a node in the Binary Search Tree."""
```

```
    def __init__(self, data):
```

```
        """Initialize a BST node."""
```

```
        self.data = data
```

```
        self.left = None
```

```
        self.right = None
```

class BST:

```
    """A simple Binary Search Tree."""
```

```
    def __init__(self):
```

```
        """Initialize an empty BST."""
```

```
        self.root = None
```

```
    def insert(self, data):
```

```
        """Insert data into the BST."""
```

```
        self.root = self._insert_recursive(self.root, data)
```

```

def _insert_recursive(self, node, data):
    """Recursively insert data into the correct position."""

    if node is None:
        return Node(data)

    if data < node.data:
        node.left = self._insert_recursive(node.left, data)

    elif data > node.data:
        node.right = self._insert_recursive(node.right, data)

    return node

def in_order(self):
    """Perform and display an in-order traversal."""
    self._in_order_recursive(self.root)
    print()

def _in_order_recursive(self, node):
    """Recursively visit left, root, and right."""
    if node is not None:
        self._in_order_recursive(node.left)
        print(node.data, end=" ")
        self._in_order_recursive(node.right)

```

```
# Testing the BST
```

```
tree = BST()
```

```
values = [50, 30, 70, 20, 40, 60, 80]
```

```
for value in values:
```

```
    tree.insert(value)
```

```
print("In-order traversal:")
```

```
tree.in_order()
```

4. Output :

```
[Running] python -u "c:\Users\manvi\OneDrive\Desktop\ai assistant coding 2\10\task-4.py"
In-order traversal:
20 30 40 50 60 70 80

[Done] exited with code=0 in 0.236 seconds
```

5. Final Justification

The BST was implemented using recursive insertion and in-order traversal. The traversal produced the values in sorted order.

Task 5 – Priority Queue

1. Question

Use AI to implement a priority queue using Python's heapq module.

2. Prompt

Create a simple Python PriorityQueue using heapq. Implement enqueue(priority), dequeue(), and display(), handle an empty queue, and add short comments plus tests.

3. Code:

```
import heapq
```

```
class PriorityQueue:
```

```
    """A simple Priority Queue using heapq."""
```

```
    def __init__(self):
```

```
        """Initialize an empty priority queue."""
```

```
        self.queue = []
```

```
    def enqueue(self, item, priority):
```

```
        """Add an item with a priority."""
```

```
        # Negative priority makes larger numbers come out first.
```

```
        heapq.heappush(self.queue, (-priority, item))
```

```
    def dequeue(self):
```

```
        """Remove and return the highest-priority item."""
```

```
        if len(self.queue) == 0:
```

```
            return None
```

```
        priority, item = heapq.heappop(self.queue)
```

```
return item
```

```
def display(self):
```

```
    """Display all items in the priority queue."""
```

```
    print("Priority Queue:", self.queue)
```

```
# Testing the Priority Queue
```

```
pq = PriorityQueue()
```

```
pq.enqueue("Normal Task", 1)
```

```
pq.enqueue("Important Task", 3)
```

```
pq.enqueue("Urgent Task", 5)
```

```
pq.display()
```

```
print("Removed:", pq.dequeue())
```

```
print("Removed:", pq.dequeue())
```

```
print("Removed:", pq.dequeue())
```

4. Output

```
[Running] python -u "c:\Users\manvi\OneDrive\Desktop\ai assistant coding 2\10\task-5.py"
Priority Queue: [(-5, 'Urgent Task'), (-1, 'Normal Task'), (-3, 'Important Task')]
Removed: Urgent Task
Removed: Important Task
Removed: Normal Task

[Done] exited with code=0 in 0.164 seconds
```

5. Final Justification

The Priority Queue was implemented using Python's `heapq` module. Items were inserted and removed according to their assigned priority.

Task 6 – Deque

1. Question

Use AI to implement a double-ended queue using `collections.deque`.

2. Prompt

Create a beginner-friendly DequeDS class using `collections.deque`. Implement insertion and removal from both ends, `display()`, empty handling, and short comments/docstrings.

3. Code:

```
from collections import deque
```

```
class DequeDS:
```

```
    """A double-ended queue using collections.deque."""
```

```
    def __init__(self):
```

```
        """Initialize an empty deque."""
```

```
        self.items = deque()
```

```
    def insert_front(self, item):
```

```
        """Insert an item at the front."""
```

```
        self.items.appendleft(item)
```

```
    def insert_rear(self, item):
```

```
        """Insert an item at the rear."""
```

```
        self.items.append(item)
```

```
    def remove_front(self):
```

```
        """Remove and return the front item."""
```

```
        if len(self.items) == 0:
```

```
    return None
```

```
    return self.items.popleft()
```

```
def remove_rear(self):
```

```
    """Remove and return the rear item."""
```

```
    if len(self.items) == 0:
```

```
        return None
```

```
    return self.items.pop()
```

```
def display(self):
```

```
    """Display the deque."""
```

```
    print("Deque:", list(self.items))
```

```
# Testing the Deque
```

```
d = DequeDS()
```

```
d.insert_front(10)
```

```
d.insert_rear(20)
```

```
d.insert_front(5)
```

```
d.display()
```

```
print("Removed from front:", d.remove_front())
```

```
print("Removed from rear:", d.remove_rear())
```

```
d.display()
```

4. Output :

```
[Running] python -u "c:\Users\manvi\OneDrive\Desktop\ai assistant coding 2\10\task-6.py"  
Deque: [5, 10, 20]  
Removed from front: 5  
Removed from rear: 20  
Deque: [10]  
  
[Done] exited with code=0 in 0.126 seconds
```

5. Final Justification

The Deque was implemented using collections.deque and tested from both ends. It supports insertion and removal at the front and rear.

Task 7 – Real-Time Application Challenge – Choose the Right Data Structure

1. Question

For the Campus Resource Management System, select a suitable data structure for each feature, justify each choice briefly, and implement one feature.

2. Prompt

For the five Campus Resource Management features, choose a suitable data structure and give a short justification for each. Then implement one selected feature as a simple Python program with comments.

3. Code:

```
class CafeteriaQueue:
```

```
    """Queue for managing cafeteria orders."""
```

```
    def __init__(self):
```

```
        """Initialize an empty order queue."""
```

```
        self.orders = []
```

```
    def add_order(self, student_name, food):
```

```
        """Add a student's order to the queue."""
```

```
        order = f"{student_name} - {food}"
```

```
        self.orders.append(order)
```

```
        print("Order added:", order)
```

```
    def serve_order(self):
```

```
        """Serve the first order in the queue."""
```

```
        if len(self.orders) == 0:
```

```
            print("No orders to serve.")
```

```
            return
```

```
        order = self.orders.pop(0)
```

```
        print("Order served:", order)
```

```
def display_orders(self):  
    """Display all pending orders."""  
    if len(self.orders) == 0:  
        print("No pending orders.")  
    else:  
        print("Pending Orders:")  
        for order in self.orders:  
            print(order)
```

Testing the Cafeteria Order Queue

```
cafeteria = CafeteriaQueue()
```

```
cafeteria.add_order("Rahul", "Burger")
```

```
cafeteria.add_order("Priya", "Pizza")
```

```
cafeteria.add_order("Arjun", "Sandwich")
```

```
print()
```

```
cafeteria.display_orders()
```

```
print()
```

```
cafeteria.serve_order()
```

```
print()
```

```
cafeteria.display_orders()
```

4. Output :

```
[Running] python -u "c:\Users\manvi\OneDrive\Desktop\ai assistant coding 2\10\task-7.py"
Order added: Rahul - Burger
Order added: Priya - Pizza
Order added: Arjun - Sandwich

Pending Orders:
Rahul - Burger
Priya - Pizza
Arjun - Sandwich

Order served: Rahul - Burger

Pending Orders:
Priya - Pizza
Arjun - Sandwich

[Done] exited with code=0 in 0.202 seconds
```

5. Final Justification

Suitable data structures were selected for the campus features based on their required operations. One feature was implemented as a working Python program with AI assistance.

Task 8 – Smart E-Commerce Platform – Data Structure Challenge

1. Question

For the Smart Online Shopping System, select a suitable data structure for each feature, justify each choice briefly, and implement one feature.

2. Prompt

For the five e-commerce features, choose a suitable data structure and give a short justification for each. Then implement one selected feature as a simple Python program with comments.

3. Code

```
class OrderQueue:
```

```
    """Queue for processing e-commerce orders."""
```

```
    def __init__(self):
```

```
        """Initialize an empty order queue."""
```

```
        self.orders = []
```

```
    def add_order(self, order_id, customer):
```

```
        """Add a new order to the queue."""
```

```
        order = f"{order_id} - {customer}"
```

```
        self.orders.append(order)
```

```
        print("Order added:", order)
```

```
    def process_order(self):
```

```
        """Process the first order in the queue."""
```

```
        if len(self.orders) == 0:
```

```
            print("No orders to process.")
```

```
            return
```

```
        order = self.orders.pop(0)
```

```
        print("Order processed:", order)
```

```
def display_orders(self):  
    """Display all pending orders."""  
    if len(self.orders) == 0:  
        print("No pending orders.")  
    else:  
        print("Pending Orders:")  
        for order in self.orders:  
            print(order)
```

Testing the Order Queue

```
order_queue = OrderQueue()
```

```
order_queue.add_order("ORD001", "Rahul")
```

```
order_queue.add_order("ORD002", "Priya")
```

```
order_queue.add_order("ORD003", "Arjun")
```

```
print()
```

```
order_queue.display_orders()
```

```
print()
```

```
order_queue.process_order()
```

```
print()
```

```
order_queue.display_orders()
```

4. Output

```
[Running] python -u "c:\Users\manvi\OneDrive\Desktop\ai assistant coding 2\10\task-8.py"
Order added: ORD001 - Rahul
Order added: ORD002 - Priya
Order added: ORD003 - Arjun

Pending Orders:
ORD001 - Rahul
ORD002 - Priya
ORD003 - Arjun

Order processed: ORD001 - Rahul

Pending Orders:
ORD002 - Priya
ORD003 - Arjun

[Done] exited with code=0 in 0.14 seconds
```

5. Final Justification

Suitable data structures were selected for the e-commerce features according to their operations. One feature was implemented as a working Python program with AI assistance.

Task 7 – Campus Resource Management: Data Structure Selection

Campus Feature	Data Structure	Reason
Student Attendance Tracking	Queue	Students can be processed in the order they enter the campus.
Event Registration	Hash Table	It provides fast search using a student or participant ID.
Library Book Borrowing	Priority Queue	Borrowing requests can be handled according to due dates or priority.
Bus Scheduling	Graph	Bus stops and routes can be represented as connected nodes and edges.
Cafeteria Order Queue	Queue	Orders are served in the same order in which students place them.

Task 8 – Smart E-Commerce Platform: Data Structure Selection

E-Commerce Feature	Data Structure	Reason
Shopping Cart Management	Deque	Products can be added or removed efficiently from either end.
Order Processing System	Queue	Orders are processed in the same order in which they are placed.
Top-Selling Products Tracker	Priority Queue	Products can be handled according to their sales priority.
Product Search Engine	Hash Table	Product IDs can be used for fast lookup.
Delivery Route Planning	Graph	Warehouses and delivery locations can be represented as connected nodes.